



Virtualización y Consolidación de Servidores

TRABAJO PRÁCTICO FINAL

Docente: Luis María Carriles

Alumno: Luciano Di Pietro – 53197

AÑO:2026

Contenido

1. INTRODUCCIÓN Y OBJETIVOS	3
2. ARQUITECTURA DE SOFTWARE Y TECNOLOGÍAS	3
2.1. Frontend (Capa de Presentación)	3
2.2. Backend (Capa de Lógica de Negocio)	3
2.3. Capa de Persistencia y Abstracción	3
3. DESPLIEGUE E INFRAESTRUCTURA	4
3.1. Topología de Red y Contenedores	7
3.2. Configuración del Proxy Reverso (Nginx)	7
4. DECISIONES DE DISEÑO Y OPTIMIZACIÓN EN RECURSOS LIMITADOS ..	7
5. MEDIDAS DE SEGURIDAD E INTEGRIDAD	7
5.1. Aislamiento de Red Interna	7
5.2. Control de Acceso del Sistema de Archivos	7
5.3. Mecanismo Criptográfico de Autenticación (Hasheo de Contraseñas)	8
5.4. Demonización y Persistencia del Servicio	8
5.5. Autenticación Sin Estado y Gestión de Sesión (JWT)	8
6. CONCLUSIONES	8

1. INTRODUCCIÓN Y OBJETIVOS

El presente documento detalla el diseño arquitectónico, las decisiones de infraestructura y las estrategias de seguridad aplicadas en el desarrollo y despliegue de un sitio web para un blog personal.

El objetivo principal del proyecto radica en la construcción de un sistema, eficiente y de alta disponibilidad, capaz de operar de manera autónoma dentro de un entorno de virtualización (Proxmox VE). El sistema no solo debía cumplir con los requerimientos funcionales de administración y publicación, sino también demostrar resiliencia ante recursos de hardware limitados y entornos donde existen múltiples contenedores de diferentes alumnos.

2. ARQUITECTURA DE SOFTWARE Y TECNOLOGÍAS

Para garantizar modularidad y escalabilidad, se adoptó una arquitectura desacoplada basada en la separación estricta de responsabilidades (Frontend, Backend y Base de Datos).

2.1. Frontend (Capa de Presentación)

Se implementó una interfaz SPA (*Single Page Application*) utilizando React. Debido a las restricciones del proyecto, se optó por un diseño de **Arquitectura de Archivo Único**.

Mediante el uso de *Babel Standalone*, los componentes de React, los estilos y la lógica de renderizado se consolidaron dinámicamente dentro de un único archivo `index.html`. Para la inclusión de recursos multimedia (imágenes de perfil o pdf de implementación), se aplicaron rutas desde la API que sirven los recursos desde el backend.

2.2. Backend (Capa de Lógica de Negocio)

El núcleo del servidor se construyó sobre Python utilizando el framework asincrónico FastAPI. Se seleccionó esta tecnología por su alto rendimiento y su capacidad nativa para manejar la concurrencia mediante *async/await*. Para la gestión de dependencias y aislamiento del entorno virtual, se utilizó el gestor de paquetes uv, optimizando los tiempos de compilación y reduciendo la huella de memoria en el disco del contenedor.

Cabe aclarar que tanto el FrontEnd como el Backend se encuentran dentro del mismo contenedor de Proxmox por un requisito propuesto por la cátedra.

2.3. Capa de Persistencia y Abstracción

Para el almacenamiento de datos se utilizó MariaDB. El acoplamiento entre el backend y el motor de base de datos se gestionó mediante SQLAlchemy como ORM (*Object-Relational Mapping*). El uso de SQLAlchemy aportó una capa de abstracción fundamental que permitió la asimetría de entornos: la utilización de SQLite en el ambiente de desarrollo local para pruebas ágiles, y la migración transparente hacia MariaDB en el ambiente de producción, manteniendo la consistencia de los tipos de datos (como la definición estricta de longitudes en campos VARCHAR exigida por el dialecto de MySQL).

3. DESPLIEGUE E INFRAESTRUCTURA

El despliegue en producción se realizó sobre la infraestructura de virtualización de la facultad, distribuyendo la carga en dos contenedores independientes basados en Linux. Se seleccionó Debian 12 como template para ambos contenedores.

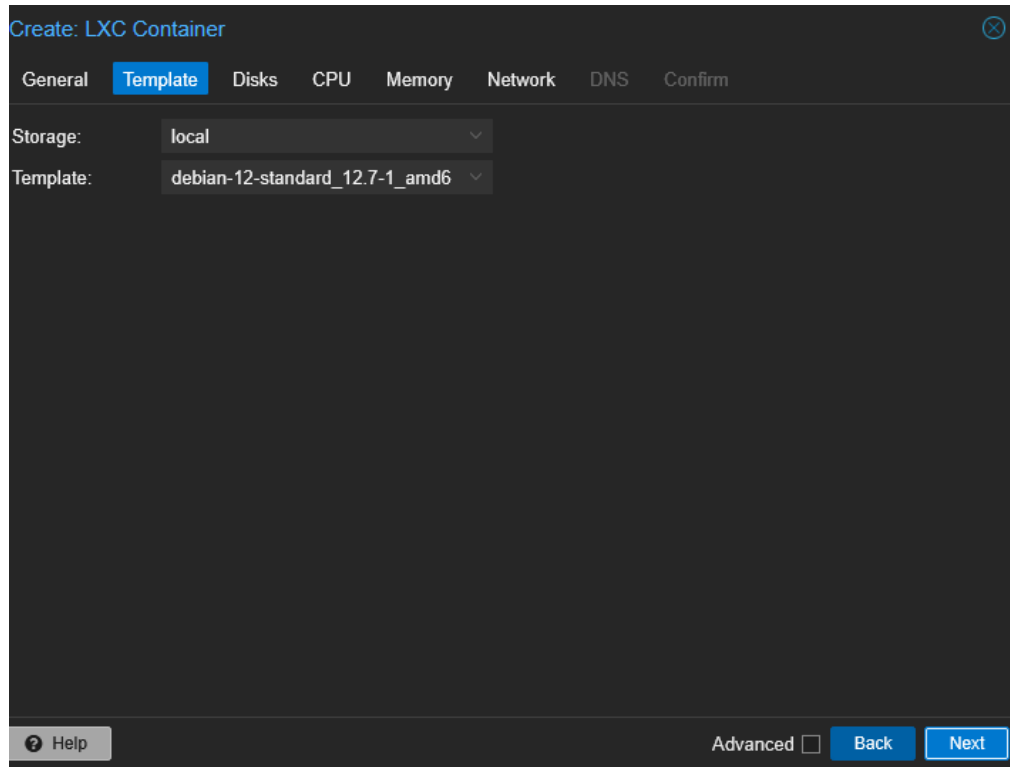


Imagen 1 - Template seleccionado para los contenedores

3.1 Configuración de los contenedores

Para cada contenedor, se asignaron 128MB de memoria RAM, contando con una capacidad muy limitada para nuestros servidores, especialmente el contenedor Web.

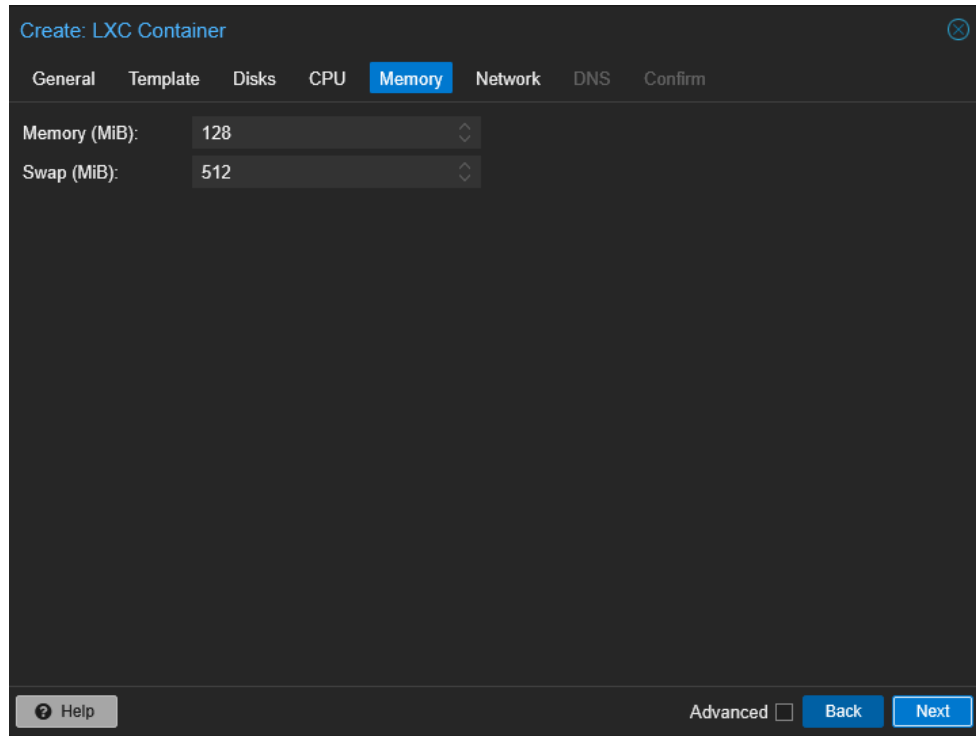


Imagen 2 – Asignación de Memoria RAM a los contenedores

Cada contenedor cuenta con 1 core para el procesamiento.

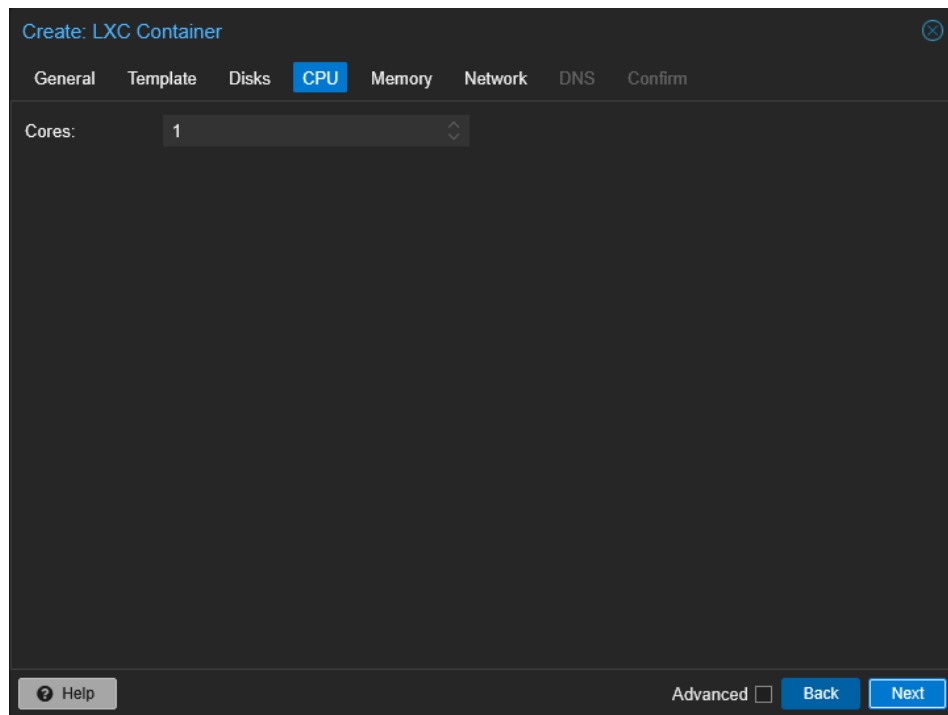
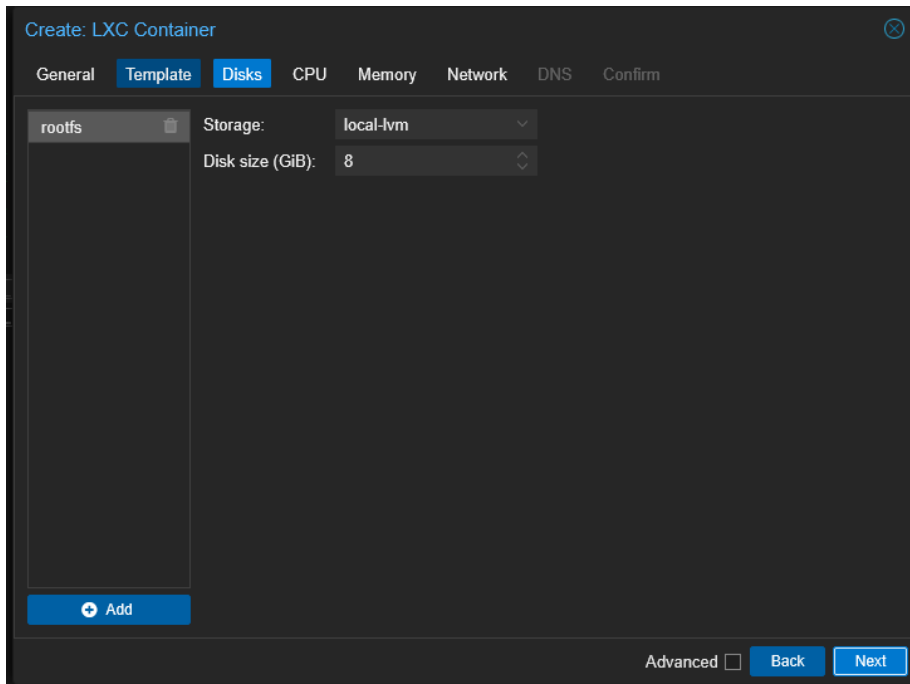


Imagen 3 – Cantidad de Cores en cada contenedor

Para el almacenamiento, podemos contar con 8 GB de disco para cada contenedor.



Create: LXC Container

General Template **Disks** CPU Memory Network DNS Confirm

Storage: local-lvm

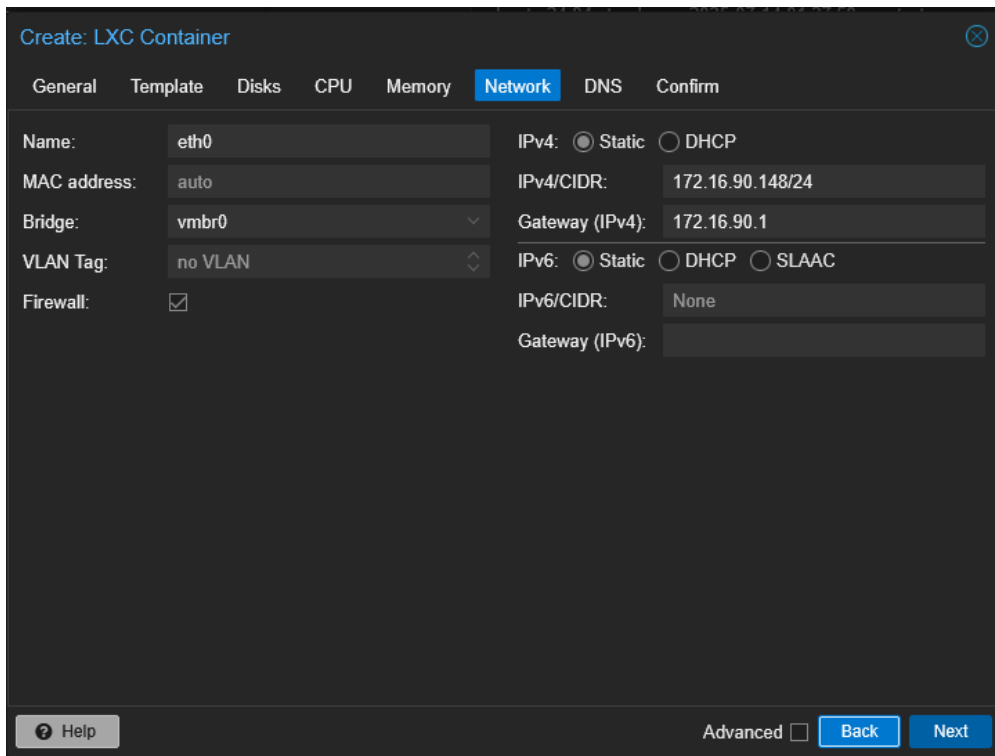
Disk size (GiB): 8

+ Add

Advanced ☐ Back Next

Imagen 4 – Almacenamiento local

Para la configuración de la red de cada contenedor, utilizamos una con IP estática, teniendo en cuenta el numero de contenedor para seleccionar el del host.



Create: LXC Container

General Template Disks CPU Memory **Network** DNS Confirm

Name: eth0 IPv4: ☒ Static ☐ DHCP

MAC address: auto IPv4/CIDR: 172.16.90.148/24

Bridge: vbr0 Gateway (IPv4): 172.16.90.1

VLAN Tag: no VLAN IPv6: ☒ Static ☐ DHCP ☐ SLAAC

Firewall: ☒ IPv6/CIDR: None

Gateway (IPv6):

? Help Advanced ☐ Back Next

Imagen 5 – Configuración de Red

3.1. Topología de Red y Contenedores

- **Contenedor Web (Frontend/Backend):** Aloja el servidor HTTP/Proxy Reverso Nginx y la API de FastAPI.
- **Contenedor de Persistencia:** Aloja de manera aislada el motor de MariaDB, impidiendo la exposición directa de la base de datos a la red.

(Insertar aquí: Diagrama de Topología de Red e Infraestructura)

3.2. Configuración del Proxy Reverso (Nginx)

Nginx se configuró para actuar como la única puerta de entrada legítima del contenedor web. Recibe las peticiones externas bajo la estructura de rutas autorizada por la universidad, sirve el archivo de presentación indexado y redirige internamente las peticiones dinámicas hacia el puerto local de la API mediante directivas proxy_pass.

4. DECISIONES DE DISEÑO Y OPTIMIZACIÓN EN RECURSOS LIMITADOS

La operación en contenedores con capacidades estrictas de CPU y memoria RAM obligó a tomar decisiones de diseño orientadas a la eficiencia:

- **Restricción de Concurrencia (Workers):** El servidor FastAPI se configuró para operar con un único worker asíncrono. Al delegar la concurrencia a la naturaleza no bloqueante de FastAPI (uso de async/await), se minimizó el consumo de memoria RAM del contenedor, manteniendo la capacidad de procesar múltiples peticiones simultáneas.

5. MEDIDAS DE SEGURIDAD E INTEGRIDAD

Debido a las características multi-usuario del servidor Proxmox, se implementó un modelo de seguridad por capas:

5.1. Aislamiento de Red Interna

Para evitar que procesos de contenedores vecinos pertenecientes a otros desarrolladores pudieran interceptar o golpear la API de forma directa, se restringió como se puede acceder al FastAPI. El servicio se configuró para escuchar estrictamente en la interfaz de loopback local (127.0.0.1:8000) y no en la interfaz abierta (0.0.0.0). Esto garantiza que ninguna petición puede llegar a la API si no ha sido previamente saneada, autorizada y redirigida por Nginx.

5.2. Control de Acceso del Sistema de Archivos

Las credenciales de acceso a la base de datos de producción y las llaves de configuración se extrajeron del código fuente y se alojaron en un archivo de variables de entorno .env (excluido del control de versiones de Git). Para mitigar el riesgo de que usuarios con acceso local al contenedor leyesen las credenciales, se modificaron los permisos del archivo, restringiendo la lectura y escritura exclusivamente al usuario propietario del proceso.

5.3. Mecanismo Criptográfico de Autenticación (Hasheo de Contraseñas)

El sistema implementa una política de almacenamiento no reversible de credenciales. Bajo ninguna circunstancia se almacenan contraseñas en texto plano dentro de la base de datos MariaDB. Cuando un administrador es registrado, la contraseña pasa por una función de derivación de claves criptográficas utilizando algoritmos de hasheo seguro. Durante el proceso de login, FastAPI intercepta la contraseña enviada por el formulario, aplica la misma función criptográfica y compara los hashes.

5.4. Demonización y Persistencia del Servicio

Para asegurar que la aplicación permanezca activa sin necesidad de mantener sesiones de terminal interactivas abiertas en la interfaz de Proxmox, la API se integró como un daemon del sistema operativo mediante Systemd. Se diseñó una unidad de servicio configurada con directivas de reinicio automático ante fallas. Esto no solo oculta los procesos de la vista de usuarios concurrentes en la terminal, sino que asegura la resiliencia y el auto-arranque automático de la plataforma ante un eventual reinicio o mantenimiento del nodo físico de Proxmox.

5.5. Autenticación Sin Estado y Gestión de Sesión (JWT)

Para el panel de administración, se implementó un mecanismo de control de acceso y mantenimiento de sesión basado en el estándar JSON Web Token (JWT).

Una vez que el servidor FastAPI valida de forma exitosa las credenciales y el hash criptográfico del administrador contra la base de datos MariaDB, emite un token firmado digitalmente con una clave secreta del servidor. La elección de JWT sobre los sistemas tradicionales de sesión (basados en cookies de servidor) responde directamente a una decisión de optimización de recursos.

6. CONCLUSIONES

La realización de este proyecto permitió aplicar de manera práctica conceptos aprendidos previamente, y otros que aprendí durante el desarrollo de este trabajo, de infraestructura, redes y seguridad en un escenario de despliegue real altamente restrictivo. Hasta este trabajo, nunca había tenido la oportunidad de desplegar en un servidor compartido una aplicación web. Las veces en las que realicé despliegues no se presentaron las dificultades de trabajar en un entorno en donde cada petición o recurso hace la diferencia.

Cambió poco mi forma de ver la implementación y despliegue, e incluso, el desarrollo. Por lo general, a la hora de elegir las tecnologías o lenguajes, no tenía en cuenta mas que mi familiaridad o el proyecto que debía construir. En este trabajo aprendí que incluso para algo básico como un blog, el servidor de producción marca mucho las decisiones a tomar. Incluso me recuerda a la materia Ing. De Software, donde veíamos mucho las diferencias de los entornos entre el local, testing y producción. Pero allí lo veíamos muy conceptualmente, ahora que lo use en práctica lo comprendo.

Todas las modificaciones y conflictos que tuve que superar para alojar mi servicio en Proxmox me hicieron entender la importancia de un ingeniero que pueda resolver las problemáticas que se presenten, diseñar una arquitectura que funcione adecuadamente en el lugar que debe funcionar y maximizar el uso que se le da a los recursos disponibles.